

SetrixDB — Apresentação do Produto

O motor de conjuntos para a era da IA na borda

Documento de *pitch*. Números reais medidos em 14/09/2026 (VPS 2 vCPU Zen4/AVX-512, Go 1.22 + cgo).

Fonte de todos os dados: `docs/RESULTADOS.md` .

1. O problema

A computação moderna virou **álgebra de conjuntos sobre IDs**: filtragem de candidatos, membership, interseção, exclusão de listas, cruzamento de sinais. Isso aparece em busca, recomendação, antifraude, telemetria, redes, bioinformática e em qualquer pipeline de IA na borda.

O hardware mudou para atender isso — ****NPU**s, SIMD largo (AVX-512/AVX10), chipsets de baixo consumo**. Os bancos de dados evoluíram para transações, analytics e busca por similaridade — e fazem isso muito bem. Mas ****nenhum deles trata "conjunto de inteiros" como cidadão de primeira classe****.

A lacuna: falta uma camada dedicada a operações de conjunto **exatas, em memória** e que **explore o hardware aritmético** ao máximo. Essa é a lacuna que o SetrixDB preenche — **ao lado** dos bancos que você já usa, não no lugar deles.

2. O que é o SetrixDB

Um **motor de conjuntos embarcável** — em memória, não-relacional, não-vetorial e puramente aritmético.

- A unidade fundamental de dados é um ID `uint64` .
- A "inteligência" da busca **reduz-se a comparações e operações sobre inteiros** —

sem ponteiros indiretos, sem hashing na consulta, sem estruturas de ponteiros.

- O conjunto é um **shard de memória contíguo** (`[]uint64`), entregue ao hardware

por **zero-copy (DMA)**.

- Projetado para **execução paralela em aceleradores** (NPUs, SIMD) em **hardware modesto**.

Duas verdades de posicionamento:

1. **Não é "mais um banco"**. É um **motor** que **coexiste** com o seu banco atual — o dado continua onde você confia; o SetrixDDB responde as perguntas de conjunto.

1. **Ele armazena conjuntos de IDs, não os payloads**. O documento/registro fica no seu banco; o SetrixDDB guarda a representação dos IDs (MPHF + bitset) e faz a álgebra em microssegundos.

Categoria que cunhamos: *Arithmetic / Set Database* (engine).

Base técnica: **MPHF** → **bitset/shard** → **kernel vetorizado** → **cluster por shard**.

3. Onde ele se encaixa (por que não substitui nada)

Cada categoria tem seu papel — e o SetrixDDB **respeita todos eles**. Ele é uma **camada a mais**:

Categoria	Papel dela	SetrixDDB é isto?
Relacional	tabelas, JOINS, SQL, transações	não — coexiste
Colunar	analytics, compressão por coluna	não — coexiste
NoSQL chave-valor	registros/documentos por chave	não — coexiste
Vetorial	similaridade aproximada por embeddings	não — é o par exato
Grafo	vértices/arestas, travessia	não — coexiste
Bitmap comprimido clássico	conjuntos comprimidos, CPU escalar	△ primo — nós somos o próximo salto
SetrixDDB	álgebra exata de conjuntos de IDs em RAM, vetorizada	▯ é isso

Em uma frase: o SetrixDDB não guarda *documentos*, *linhas* nem *vetores* — ele guarda **conjuntos**. E responde a perguntas sobre eles com **aritmética vetorizada**, exata e em microssegundos, mesmo em hardware pequeno.

4. Como funciona (arquitetura em 4 passos)

...

texto/ID cru → MPHf (IDs densos, 0 colisão) → Shard/bitset em RAM

| zero-copy (DMA)

▼

Kernel aritmético vetorizado (AVX-512 / NPU)

|

▼

Cluster por shard (consistent hash ring, TCP zero-copy)

...

1. **ID único** — MPHf (Perfect Hash) próprio: 0 colisão, ~4 bits/chave, lookup $O(1)$ ~118 ns.
2. **Representação** — bitset denso, SparseSet ou **híbrido** (faixas quentes densas + cauda esparsa).
3. **Kernel** — `vpandq` + `vpopcntq` (AVX-512), com *dispatch* em runtime e fallback escalar

(cobre AVX10.2 e CPUs sem AVX-512).

1. **Escala** — shards distribuídos por **consistent hash ring**; consulta é *broadcast* zero-copy,

com modo **conjuntos armazenados** (só o nome do set trafega na rede).

5. Diferenciais — números reais (não promessa)

Frente	SetrixDDB	Estado da arte	Ganho
ID único (MPHF CHD v2, n=50M)	4,03 bits/chave · 0 colisão · 118 ns	—	~ piso teórico (1,44) na fronteira prática
Interseção densa (1M×1M)	6 µs (AVX-512)	Roaring 148 µs	~24×
Interseção esparsa (universo 2 ²⁶)	303 µs	Roaring64 9,26 ms	~30×
Membership	paridade c/ map , exato	map 22,3 B/chave	2,2× menos memória
Memória (universo 2 ³⁶ , 4M chaves)	1,73 MB (híbrido)	bitset denso 8,59 GB	memória ∝ dados, não ∝ universo
Distribuído (link lento WG)	19,4 ms (sets armazenados)	364 ms (ad-hoc)	19×
Topologia (entra 1 nó, 16 nós)	7% remapeado (ideal 6%)	módulo: ~94%	rebalanceamento incremental

Correção idêntica entre local e distribuído em todos os testes (`go test ./...` verde:

`adbb , cluster , mphf , simd`).

6. Onde isso vale dinheiro (casos de uso)

- **Busca & filtragem em escala** — cruzamento de listas de IDs (`A ∩ B`), facetas, dedup.
- **Recomendação / feature store** — geração de candidatos em microssegundos.
- **Antifraude & segurança** — listas de bloqueio, correlação de sinais, ACLs.
- **Telemetria & observabilidade** — séries por ID (mesmo território do `cloudx-tsdb`).
- **Rede & nuvem** — tabelas de rotas, VPCs, políticas L4 (a própria **CloudVerse**).
- **Borda / Edge AI** — latência e energia mínimas em SBCs e NPUs.
- **Busca exata em dicionários** — ex.: os ~50 mi de termos validados.
- **Bioinformática / DSP** — k-mers, correlação de sinais.

7. Estado atual (o que já é verdade)

- **POC em Go**, ~28 arquivos, ~13 famílias de teste — **todos os testes verdes**.
- Resultados **medidos e reproduzíveis** (`go test ./... + cmd/{setbench,vsbench,sparsebench,shardbench,mphfbench,ringbench}`).
- **Distribuído validado de verdade**: 4 nós locais e **VPS** ↔ **nó remoto via WireGuard**.

- Documentação completa (README , ESPECIFICACAO , ARQUITETURA , RESULTADOS) + PDF consolidado.
- **Sem dependências de terceiros** no núcleo; MPHF, bitset, ring e cluster escritos do zero.

8. Roadmap

Fase	Entrega
▣ Feito	MPHF, bitset/SparseSet/híbrido, kernel AVX-512, sharding, cluster over-the-wire, hash ring
Agora	Testes de escala em nuvem (CloudVerse) — cluster real multi-nó
Próximo	Driver de NPU (DMA zero-copy + pinning), protocolo UDP/gRPC, benchmarks de energia (J/busca) em SBC
Produto	SDK embarcável (Go/C) + serviço de cluster gerenciado (edge-first)

9. Por que é diferenciado

1. **Categoria própria** — não compete de frente com SQL/NoSQL/vetorial; abre a categoria *arithmetic/set DB*.
2. **Ganha do estado da arte no que importa** — 24–30× na operação central (interseção).
3. **Feito para o hardware de hoje** — SIMD/NPU, não disco.
4. **Escala sem reescrever** — sharding + ring + conjuntos armazenados.
5. **Compatível com o mundo real** — foi feito para **coexistir** com qualquer stack; é um *engine* de conjuntos, não uma substituição.

Pitch em uma linha: *"O SetrixDB responde interseções e membership sobre bilhões de IDs em microssegundos, exato e em qualquer hardware — o motor de conjuntos que a era da IA na borda estava esperando."*